

대표 포트폴리오

에-라잇

AI Writer System — 장편 창작을 위한 글쓰기 운영체제

장편에서 무너지는 것은 문장력이 아니라
일관성이다.
기억을 설계하고, AI가 쓴 것은 검증된 뒤에만
사실이 되게 만들었습니다.

역할
제품 오너 · 기획 · AI 오케스트레이션
(1인)

기간
2026.06 ~ 진행 중
개인 프로젝트

작업 방식
오너 1인 + AI 작업 세션
(Claude Code · Codex)

작성
박요한
github.com/entangelk/ai_writte_system



시장에서 문제를 찾고, 기억 구조로 풀고, AI를 규칙과 검증으로 다뤘습니다

1 시장

1조 3,500억 원 웹소설 시장

연재형 장편이 중심이고, 작가들은 이미 생성형 AI를 쓰고 있다. 하지만 본문 집필에는 쓰지 못한다.

2 문제

AI는 장편의 앞뒤를 기억하지 못한다

설정 오류·회차 간 모순으로 AI 본문은 대부분 폐기된다. 긴 문맥의 중간을 놓치는 것은 모델의 구조적 한계다.

3 해법

생성기가 아니라 기억 시스템

원고에서 기억 후보를 뽑아 사람이 확정하고, 쓸 때마다 필요한 기억만 근거와 함께 검색해 모델에 준다.

4 오케스트레이션

AI는 제안하고, 결정은 사람이, 검증은 다른 세션이

지침서로 AI 작업 방식을 고정하고, 결정 브리프·독립 검증·정본 문서로 1인 규모의 개발을 통제했다.

1,266

커밋 (2026.06.24 ~ 09.14)

118건

구현 전에 쓴 결정 브리프

315건

독립 검증 기록 (73일치)

30%

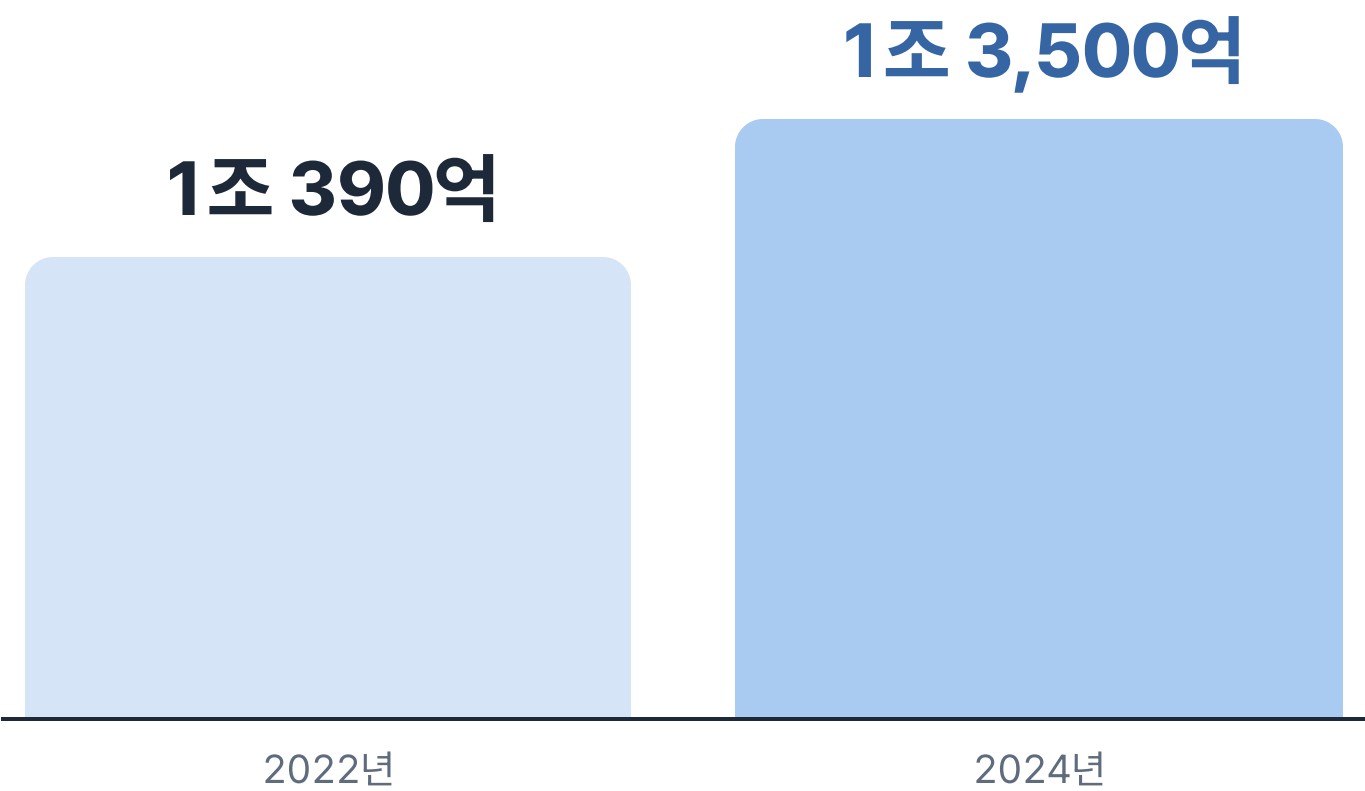
조건부 합격 비율
검증이 형식이 아니라는 증거

3,075

통과하는 백엔드 회귀 테스트

웹소설은 2년 새 30% 커진 1조 3,500억 원 시장이고, 그 중심은 수백 회차의 장편 연재입니다

■ 국내 웹소설 산업 규모



2년 동안 **3,110억 원 증가(+30%)**. 웹툰·영화·게임으로 확장되는 **원천 IP**로서 가치가 커지고 있습니다.

79%

이용자 유료 결제 경험
(1회 평균 8,032원)

800명

실태조사에 참여한 창작자
(CP 152곳 · 플랫폼 10곳)

■ 장편 연재의 작업량

- 회차당 통상 5,000~6,000자, 주 5~7회 연재
- 100화면 **50만 자** 이상 — 작가 한 명이 기억해야 하는 설정·인물·복선이 회차마다 쌓인다
- 독자가 매주 결제하는 구조라, 앞뒤가 안 맞는 회차는 곧바로 이탈과 댓글로 돌아온다

기회 — 시장은 크고 성장 중이며 작업량이 거대하다. 작가의 부담 중 AI가 줄일 수 있는 부분이 분명히 있다.

작가들은 이미 AI를 쓰지만, 본문 집필에서는 버립니다.

원인은 문장력이 아니라 연속성입니다

AI가 이미 잘 쓰이는 곳

- 기획 — 설정 정리, 세계관 확장
- 퇴고 — 맞춤법, 논리, 설정 충돌 점검
- 아이디어 — 플롯 제안, 캐릭터 초안

"집필 단계에서 AI가 생성한 본문은 세계관 오류나 장르·시대 맥락 어긋남, 부족한 분량, **회차 간 내용 유출** 등으로 인해 대부분 폐기되거나 전면 재작성되었다."

『웹소설 작가 지망생의 생성형 AI 활용 경험 탐색』, 한국출판학연구, 2026.06

본문에서 무너지는 이유 — 모델은 긴 문맥의 중간을 놓친다



긴 입력에서 필요한 정보가 **중간에 있을수록 성능이 떨어진다(U자형)**. 장편으로 옮기면 중간 회차의 인물 특성·복선을 잊어 행동과 전개가 어긋난다는 뜻이다.

- 문맥 창을 늘려도 해결되지 않는다 — 넣는 것과 쓰는 것은 다르다
- 범용 챗봇은 대화창을 벗어나면 아무것도 기억하지 못한다

결론 — 필요한 것은 더 좋은 생성 모델이 아니라, **필요한 기억만 골라 근거와 함께 넣어 주는 구조**다. 이것이 제품의 출발점이 됐다.

무엇을 만드는가 — 생성기가 아니라 작가의 기억을 관리하는 글쓰기 운영체제

- 장편에서 실제로 깨지는 것
 - 인물의 말투가 3장과 17장에서 다르다
 - 20장 전에 죽은 인물이 되살아난다
 - 작가 자신도 "그때 그 설정이 뭐였더라"를 찾지 못한다

한 문장 정의

원고·설정·세계관·문체를 장기 기억으로 쌓고, 글을 쓰는 시점마다 필요한 기억만 근거와 함께 검색해 모델에 주는 개인 창작 메모리 시스템.

출발점 — 실무에서 설계한 검증형 RAG 구조와 로컬 LLM 게이트웨이 구조를 창작 도메인에 응용했습니다.

포지셔닝 — 범용 챗봇과 무엇이 다른가

관점	범용 AI 챗봇	에-라잇
기억	대화창 안에서만 유지	작품 단위 장기 기억, 버전으로 쌓임
AI 출력의 지위	답변이 곧 결과	모든 출력은 후보, 검토 뒤에만 기억이 됨
근거	출처를 알 수 없음	모든 기억이 원문 위치를 가리킴
긴 문맥	전부 넣고 모델에 맡김	필요한 조각만 검색해 예산 안에서 조립
품질 통제	사용자가 눈으로 확인	생성 뒤 게이트가 설정·시점·문체를 판정
작품 간 섞임	통제 불가	프로젝트 격리를 전 계층에서 강제

기획 단계에서 원칙을 먼저 못박고, 만들면서 범위를 근거 있게 줄였습니다

■ 전 단계를 구축하는 제품 원칙 4개

AI 출력은 정본이 아니다

생성·분석 결과는 전부 후보로 남는다. "AI가 쓴 것이 곧 사실"이 되는 순간 기억이 오염되고, 그 오염은 다음 생성의 입력이 되어 복리로 커진다.

기억은 덮어쓰지 않는다

append-only로 버전을 쌓는다. 잘못된 갱신이 과거를 지우지 못해야 AI에게 쓰기를 맡길 수 있다.

모든 기억에 근거 포인터

원문 위치까지 되짚을 수 있어야 작가가 AI의 판단을 검증할 수 있다. 검증할 수 없는 기억은 신뢰 대상이 아니다.

작품끼리 섞이지 않는다

다른 작품의 설정이 섞이는 것은 품질 저하가 아니라 제품의 실패다. 저장·검색·게이트 전 계층에서 격리한다.

■ 원안(2026.06)에서 바꾼 범위와 그 이유

항목	원안	현재	판단 근거
기억 추출	인물·사건·장소·복선·관계 5종	인물·사건·열린 질문 관찰 3종	과잉 추론 위험이 큰 항목부터 줄이고, 복선은 "열린 질문"으로 흡수
품질 게이트	연속성·시점·복선 게이트 각각	하나의 Writing Gate, 판정 5종	한 번의 판정이 네 유형의 지적을 내는 편이 비용·일관성 모두 유리
문체	과거 글에서 학습	작가가 규칙으로 선언	학습은 데이터·검증 비용이 크다. 선언은 즉시 쓰이고 틀려도 되돌리기 쉽다
사용자	단일 사용자	다중 사용자·소유권 격리	오너 결정(2026.07.26)으로 유예 만료. 기존 격리 위에 소유권을 얹음
일부러 안 함	공동 집필 · 관계 그래프·타임라인 · 자기 가입 화면 — 각각 "무엇이 오면 여는가" 트리거를 붙인 백로그로 분리		

기억이 쌓이는 쪽과 쓰이는 쪽이 하나의 루프로 닫힙니다

LLM은 9곳에서만 불리고 전부 게이트웨이 뒤에 있습니다. 검색 검증·판정 규칙·토큰 예산은 LLM이 아니라 결정론 코드가 가져서, 모델을 바꿔도 안전성은 그대로입니다.

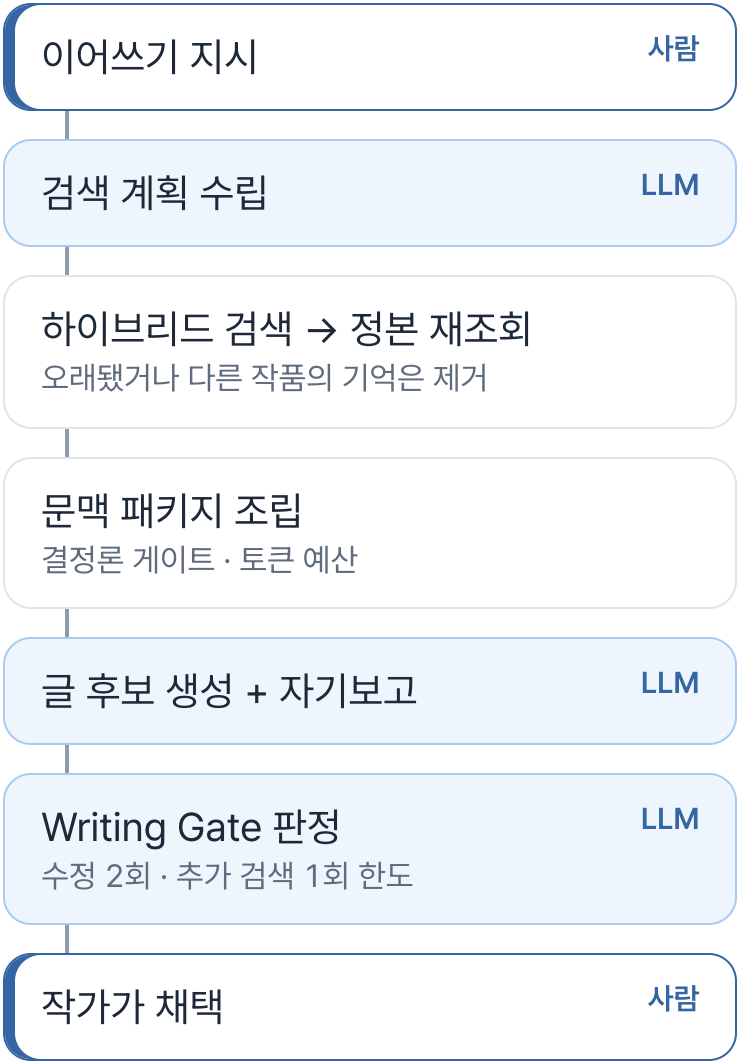
① 기억이 쌓이는 쪽

원고 → 후보 → 사람이 확정



② 기억이 쓰이는 쪽

필요한 기억만 골라 생성



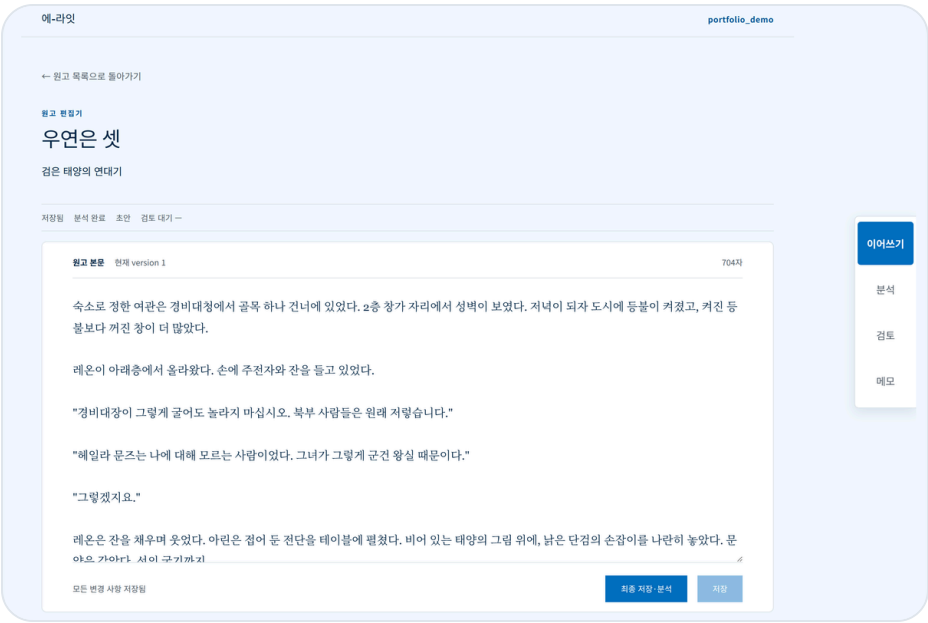
채택 원고는
다시 ①로

☐ 결정론 코드 ☒ LLM 호출 ☐ 사람의 결정

설계 판단 셋

- **벡터 DB는 진실이 아니라 캐시**
검색 결과를 그대로 쓰지 않고 MongoDB 정본을 다시 읽어 버전·해시를 확인한다. 색인은 언제든지 다시 만들 수 있다.
- **게이트는 판정만, 결정은 서버가**
LLM 게이트는 지적을 내고, 최종 판정(통과·수정·추가 검색·사람 검토·차단)은 서버가 규칙으로 파생한다.
- **루프에는 반드시 한도**
수정 재생성 2회, 추가 검색 1회, 게이트 3회. 한도는 실제 12B 모델로 단계별 비용을 재서 정했다.
- **창을 넘는 요청은 부르기 전에 거절**
예산을 모델 창에서 유도하고, 넘으면 모델 호출 없이 400으로 돌려준다.

원고를 쓰고, AI가 뽑은 기억을 근거와 함께 확정하고, AI가 일하는 모습을 지표로 봅니다



원고 작업공간

저장하면 스냅샷이 남고 분석이 돈다. 오른쪽 서랍에서 이어쓰기·분석·검토·메모를 연다.

한 번에 뜬다

FastAPI · React/TS · MongoDB · ChromaDB · Elasticsearch · LLM 게이트웨이까지 **docker compose up** 하나로 전 스택이 뜬다.



기억 후보 검토

AI가 추출한 인물 관찰을 원문 근거와 나란히 보여 준다. 승인해야만 정보 기억이 된다.

모델을 갈아 끼운다

로컬 llama.cpp(Gemma 12B)와 외부 API(Gemini 등)를 **게이트웨이 뒤에서 교체**한다. 키 회전·모델 폴백까지 여기서 통제.



LLM 호출 지표

호출부별 성공·실패, 토큰, 게이트 판정 점수. 실패한 호출도 센다.

제품으로 필요한 것

계정·세션·소유권 격리, 관리자 표면 분리, 요청 한도, 탈퇴 30일 뒤 영구 파기, 약관·방침 1.0.

AI에게 "만들어 줘"라고 하지 않았습니다. 역할을 나누고, 결정과 검증이 흐르는 길을 설계했습니다

오너

박요한

제품 방향과 모든 되돌리기 어려운 결정

AI가 올린 결정 브리프에서 선택지를 고른다

결정 원문과 이유를 일일 로그에 남긴다

지침서(작업 규칙)를 쓰고 사고가 날 때마다 고친다

구현 세션

Claude Code · Codex

같은 지침서(CLAUDE.md · AGENTS.md)를 따른다

결정이 필요하면 추측하지 않고 브리프를 쓰고 멈춘다

양방향 회귀 가드와 함께 구현하고 체크포인트 커밋

합정과 다음 할 일을 인수인계 문서에 남긴다

검증 세션

구현하지 않은 다른 세션

구현자의 맥락을 물려받지 않고 반증을 시도한다

고친 것을 되돌려 테스트가 다시 실패하는지 확인 (뮤테이션)

판정은 합격 · 조건부 합격 · 불합격 셋뿐

조건은 후속 커밋으로 달고 다른 세션이 승격한다

① 결정 브리프

118건

선택지 표 · 추천 · 유예 항목을 쓰고 멈춘다

② 오너 결정

80일

일일 로그에 결정 원문과 이유 기록

③ 구현 + 회귀 가드

3,075

결함 재현과 과잉 교정 양쪽에서 실패하는 테스트

④ 독립 검증

315건

다른 세션이 뮤테이션으로 반증 시도

⑤ 정보 개정

v1.8.69

계약 정보 버전과 변경 이유를 전량 보존

왜 이렇게까지 하는가 — AI가 쓴 코드는 "테스트가 초록색"만으로 믿을 수 없습니다. 초록색은 지금 있는 테스트가 통과한다는 뜻일 뿐, **그 테스트가 올바른 것을 잡는지는** 말해 주지 않습니다.

AI의 실수는 주의를 당부해서는 반복됩니다. 사고가 날 때마다 실행하는 규칙으로 바꿨습니다

실제로 일어난 일	관측한 사실	지침서에 넣은 규칙
조용한 선택	AI가 아키텍처·계약 값을 스스로 골라 구현하면, 오너가 고르지 않은 길에 묶인다	결정이 필요하면 구현을 멈추고 결정 브리프를 쓴다 — 선택지를 미리 거르지 않고, 추천과 이유, 유예 항목까지 단, 명세·선례·30초 grep으로 답이 나오는 질문에는 브리프를 만들지 않는다
한쪽만 잠근 테스트	원래 결함은 잡지만 과잉 교정은 통과시키는 회귀 테스트가 결함을 "정상"으로 고정하고 있었다	회귀 가드는 양방향 — 결함을 되살려도 실패, 과잉 교정으로 정상 경로를 깨도 실패
되돌리다 코드 유실	뮤테이션을 복원하려고 git checkout 하다가 커밋 안 된 작업이 사라진 사고 9건, 그중 8건은 방금 짠 코드를 만든 구현자 본인	"조심하라"가 아니라 실행하는 관문 — 첫 뮤테이션 전 git status가 비어 있어야 한다. 커밋 → 변이 → 복원 → 확인 순서 고정
판정 문구 111가지	검증 기록 222건에서 판정 표현이 111가지로 갈려, 집계가 5건·4건씩 틀리고 불합격 1건이 일주일간 누락됐다	판정 첫 줄은 합격·조건부 합격·불합격 세 단어로만. 인덱스와 판정 분포는 테스트가 디스크와 대조해 잠근다
숫자 주장	문서의 테스트 수·건수가 실제와 어긋나기 쉽다	숫자에는 기준 시점과 재현 명령을 붙이고, 검증자는 보고된 숫자를 직접 다시 쟀다
push와 브랜치	AI 세션이 원격 저장소를 바꾸면 오너가 모르는 상태가 생긴다	AI는 push하지 않는다. 브랜치도 요청이 있을 때만. 체크포인트 커밋은 기본

저장소가 LAN에 인증 없이 열려 있었다.

AI의 실측이 추천을 바꾸고, 결정은 제가 내렸습니다

■ AI가 올린 4지선다 (결정 브리프 D8-7)

선택지	설명	장점	단점
A 노출면 축소	저장소 포트를 127.0.0.1에만 바인드	위험 즉시 0, 코드·시크릿 불필요	인증이 아니라 회피
B 자격증명 도입	Mongo keyfile·ES·Chroma 인증	진짜 인증	머신마다 시크릿 셋업, "compose up만으로 뜯다"가 깨짐
C A 먼저, B는 배포 때	지금 위험을 없애고 비용은 필요할 때	재작업 없음, B 설계는 브리프로 보존	완료 조건을 "인증"에서 "노출 없음"으로 재정의
D 하지 않음	현행 유지	비용 0	무인증 저장소가 계속 열림

선택지를 미리 거르지 않는다 — 오너가 AI가 버린 줄에서 가치를 볼 수도 있기 때문입니다.

■ 어떻게 결정이 났나

착수 전 실측	게시된 포트 7개 전부 0.0.0.0. Mongo 자격증명 배선 지점 13곳을 AST로 전수 확인 — 코드 변경은 0줄이면 된다
추천을 바꾼 사실	인증을 켜 복제셋 Mongo가 keyfile 을 강제한다는 것을 컨테이너로 직접 확인. "비밀번호 하나"가 아니라 커밋할 수 없는 시크릿 파일이 머신마다 필요했다
오너 결정	C 채택(2026-08-02). 로컬 1인·머신 3대 단계에서 같은 위험에 더 싼 해법
시행	compose 3개 파일만 수정, 코드 0줄. compose 파일을 읽어 노출을 강제하는 전수 회귀 테스트 추가
유예의 조건	B는 각하가 아니라 유예 — 원격 배포를 시작하는 날 이 브리프의 표가 그대로 실행 계획이 된다

검증 10건 중 3건은 깨끗하게 돌아오지 않았습니다. 그 비율이 절차가 형식이 아니라는 증거입니다

315건의 독립 검증 판정은 **합격 216 · 조건부 합격 94 · 불합격 5**입니다. 조건부 합격은 "합격이되 달아야 할 조건이 있다"는 뜻이고, 각 조건은 후속 커밋에서 달힌 뒤 다른 세션이 승격합니다. 아래는 테스트만으로는 보이지 않았던 세 가지입니다.



"시행 완료"가 런타임에서는 거짓이었다

포트 노출 차단이 compose 파일에서는 참이었다. 하지만 이미 만들어진 컨테이너가 옛 포트 매핑을 그대로 들고 있었다.

잡은 것은 테스트 스위트가 아니라 검증자가 직접 친 **docker ps**였다.

파일 참 ≠ 런타임 참

verifications/2026-08-02

테스트가 결함을 "정상"으로 고정하고 있었다

유료 API 중복 제출을 막는 잠금이, 만료된 잠금으로 거짓 차단을 하거나 저장되지 않은 성공을 돌려줄 수 있었다.

독립 검증의 **불합격**으로 발견됐고, 수정은 코드보다 **테스트 기대치를 고치는** 일에서 시작했다.

불합격 → 계약 개정

verifications/2026-08-03 · 재검 08-04

추측 대신 재서 프롬프트를 73% 줄였다

실제 12B 모델에서 긴 보고서 생성이 4번 중 4번 실패. 토큰이 어디서 나가는지부터 재니, 문맥 항목마다 붙는 근거 포인터 JSON이 원인이었다.

모델에게 덜 주는 대신 **모델이 덜 쓰게** — 포인터 대신 번호를 쓰고 매핑은 서버가 한다.

11,841 → 3,216 토큰

헤드룸 -1,665 → +7,024 · 실모델 3/3 성공

감으로 프롬프트를 바꾸지 않도록, 이미 있는 판단 AI의 결정을 품질 지표로 재활용했습니다

■ 운영 질문 → 지표 → 내리는 결정

지표	답하는 질문	유발하는 결정
파이프라인 카운트	얼마나 일하고 얼마나 되돌리나	재시도·미수렴이 잦으면 프롬프트·모델·검색 점검
게이트 품질점수	생성 AI가 얼마나 잘 썼나	프롬프트·모델 교체 전후 비교
호출부별 결과율	어느 단계가 불안정한가	가장 흔들리는 단계부터 손댄다
토큰·지연	원고 한 건의 비용과 응답성	비용 예측, 문맥 예산 조정

실패한 호출도 센다 — 성공만 세면 성공률은 영원히 100%가 됩니다. 9개 호출부 전부가 같은 형식의 감사 레코드를 남기고, 기록이 실패해도 사용자 요청은 영향받지 않게 격리했습니다.

■ 게이트 판정을 0~1 점수로



■ **한계를 함께 적었다**

게이트는 가장 심각한 지적 하나로 판정하므로, 지적의 개수·조합은 점수에서 사라집니다. 이것은 버그가 아니라 **의도한 1차 근사**이고, 게이트가 심각도를 반영하는 정밀화와 "게이트 판정 vs 작가의 실제 채택" 캘리브레이션을 로드맵으로 남겼습니다.

큰 관측 인프라를 새로 짓지 않았다 — 파이프라인이 이미 쓰는 저장소 위에 표준 레코드 한 종류를 얹어 낮은 비용으로 시작했습니다.

1인과 AI 세션으로 제품 규모의 시스템을 만들었고, 아직 증명하지 못한 것도 그대로 적습니다

1,266

커밋 · 2026.06.24 ~ 09.14

107

API operation · 인증 tier 전수 가드

약 6.5만

코드 줄 수 · Python 4.4만 + TS 2.2만

9곳

LLM 호출부 · 전부 게이트웨이 뒤, 전부 계측

■ 정직한 한계

– 작품 품질 데이터는 아직 없다

지금의 숫자는 전부 시스템 지표다. "원고가 실제로 좋아지는가"는 실사용 관찰로 확인할 몫

– 실사용은 착수 선언 단계

2026-08-23 착수 선언. 운영은 로컬 1인, 원격 다중 호스트 배포는 유예

– 미룬 화면과 기능

인물 카드·타임라인·관계 그래프, 뉴럴 리랭커 품질 평가

■ AI 프로젝트 빌더로서 배운 것

AI는 제안하고, 규칙이 판정하고, 사람이 결정한다

제품 안의 LLM에도, 개발하는 AI 세션에도 같은 원칙이 통했다. 생성보다 검증과 권한 설계가 품질을 만든다.

지침은 당부가 아니라 실행하는 관문이어야 한다

같은 사고가 아홉 번 반복된 뒤에야 알았다. 규칙을 문서로 두고, 사고가 날 때마다 규칙을 고치는 것이 오케스트레이션이다.

숫자는 추정이 아니라 측정에서 온다

루프 한도, 프롬프트 예산, 검증 분포 — 결정에 쓴 숫자는 전부 다시 짤 수 있게 남겼다.